DEEP PRODUCT ANALYSIS

CataloGlobe

Analisi approfondita del prodotto, dell'architettura e della visione di piattaforma

> *"Cataloghi digitali dinamici che sanno dove, quando e cosa mostrare — automaticamente."*

Marzo 2026 · Documento interno · v1.0

Indice

01 Panoramica di alto livello

02 Problema e Soluzione

03 Concetti fondamentali

04 Come funziona il sistema

05 Funzionalità chiave

06 Architettura e insight tecnici

07 Cosa rende speciale CataloGlobe

08 Rischi e aree incomplete

SEZIONE 01

Panoramica di alto livello

CataloGlobe è una **piattaforma SaaS multi-tenant** per la gestione di cataloghi digitali pensata per ristoranti, hotel, retail e altri settori. A differenza dei generatori di menu statici, CataloGlobe tratta i cataloghi come **sistemi viventi**.

Un singolo database prodotti alimenta molteplici cataloghi, che vengono assegnati a sedi specifiche tramite regole di programmazione basate sul tempo. Una catena di ristoranti può mostrare il menu pranzo dalle 11 alle 15, passare al catalogo cena alle 18 e promuovere un bundle weekend — il tutto senza intervento manuale.

> **La separazione fondamentale:** La piattaforma separa **contenuto** (prodotti, cataloghi, highlight) da **presentazione** (stili, token) e **distribuzione** (sedi, programmazione). Ogni livello è indipendentemente modificabile.

Prodotti (cosa) × Cataloghi (come organizzati) × Stili (come appare) × Programmazione (dove + quando) =

SEZIONE 02

Problema e Soluzione

Il problema

Le aziende multi-sede affrontano un incubo di content management:

- **Fatica del menu:** Aggiornare prezzi, prodotti o promozioni su 5, 50 o 500 sedi è manuale, soggetto a errori e lento.
- **Cecità al contesto:** Un singolo menu statico non può riflettere che la Sede A ha uno speciale a pranzo, la Sede B un bundle weekend e la Sede C disabilita certi prodotti per problemi di fornitura.
- **Frammentazione del design:** Ogni sede finisce con branding inconsistente — font diversi, layout diversi, template obsoleti.
- **Sprawl di strumenti:** Le aziende combinano Canva per il design, fogli di calcolo per i prezzi, WhatsApp per la distribuzione e un website builder. Niente è connesso.

Perché le soluzioni esistenti non bastano

SOLUZIONE	LIMITI
Generatori PDF (Canva, MustHaveMenus)	Output statico. Nessuna programmazione, nessuna multi-sede, nessun database prodotti.
QR Menu SaaS (Menufy, GloriaFood)	Solo ristoranti, liste piatte, nessun catalogo strutturato, nessuno scheduling.
CMS generici (WordPress + WooCommerce)	Troppo generici. Nessun concetto nativo di sedi, programmazione o isolamento tenant.

L'approccio CataloGlobe

CataloGlobe introduce una **separazione delle responsabilità** che nessuno di questi strumenti offre:

LIVELLO	RESPONSABILITÀ	ESEMPIO
Contenuto	Prodotti, varianti, gruppi opzione, attributi	"Pizza Margherita, taglie S/M/L, add-on: mozzarella extra"
Struttura	Cataloghi con categorie gerarchiche (3 livelli)	"Menu Pranzo → Antipasti → Insalate"
Presentazione	Token di stile con versionamento immutabile	"Navigazione a pill, card con immagine a sinistra, colori brand"
Distribuzione	Regole di scheduling per sedi/gruppi	"Mostra Menu Pranzo a Via Roma dalle 11–15, Lun–Ven"

LIVELLO	RESPONSABILITÀ	ESEMPIO
Promozione	Contenuti in evidenza con slot flessibili	"Hero banner: Bundle Brunch Weekend a €19.90"

SEZIONE 03

Concetti fondamentali

Tenant (Azienda / Brand)

L'unità organizzativa di primo livello — un'azienda o un brand. Ogni dato in CataloGlobe è **tenant-scoped**. Un tenant possiede tutti i prodotti, cataloghi, stili, programmazioni e sedi.

- `vertical_type`: ristorante, bar, retail, hotel o generico
- Supporta ruoli `owner`, `admin` e `member` con sistema di inviti tramite token
- Soft-delete con 30 giorni di grazia prima della cancellazione definitiva
- Gestione logo tramite RPC sicura (SECURITY DEFINER)

Attività / Sede (Activity)

Una sede fisica appartenente a un tenant. Le attività sono gli **endpoint di distribuzione** — il "dove" della consegna dei contenuti.

- Slug unico per URL pubblico (`/collections/{slug}`)
- Supporto immagini di copertina in Supabase Storage
- Raggruppabili in `activity_groups` per targeting massivo
- Stato: attivo/inattivo

Prodotto (Product)

L'unità atomica di contenuto. I prodotti esistono **una volta** e vengono referenziati ovunque. Un aggiornamento di prezzo si propaga a tutti i cataloghi (a meno che non sia sovrascritto da una regola).

CARATTERISTICA	DETTAGLIO
Varianti	Prodotti figli via `parent_product_id` (es. "Margherita → Classica, Bufala"). No nested variants.
Gruppi opzione	`PRIMARY_PRICE` (taglie con prezzo assoluto) e `ADDON` (modificatori con prezzo delta)
Attributi	Tipizzati (text/number/boolean/json). Platform attrs = `tenant_id NULL`
Allergeni e ingredienti	Collegati per compliance normativa
fromPrice	Minimo tra le opzioni PRIMARY_PRICE, mostrato come "da X€"

Catalogo (Catalog)

Una collezione strutturata di prodotti organizzati in categorie gerarchiche. I cataloghi sono il **livello strutturale**.

- Fino a **3 livelli** di gerarchia (es. "Food → Antipasti → Antipasti Freddi")
- Ordinamento esplicito tramite `sort_order`
- Constraint UNIQUE: un prodotto può apparire una sola volta per categoria
- Drag-and-drop con `@dnd-kit`
- **Nessun riferimento diretto allo stile** — il binding avviene nelle regole di scheduling

Contenuto in Evidenza (Featured Content)

Un blocco promozionale — banner, hero section o bundle — che si inserisce nelle pagine catalogo come **contenuto overlay**.

- **Slot:** `hero` (top), `before_catalog`, `after_catalog`
- **Modalità prezzo:** `none` (solo editoriale), `per_item` (somma prodotti), `bundle` (prezzo fisso)
- **Campi:** titolo, sottotitolo, descrizione, media, CTA (testo + URL)
- **Stato:** draft / published

Programmazione / Scheduling

Il motore a regole che controlla **cosa** viene mostrato, **dove**, **quando** e **come** — il sistema nervoso della piattaforma e il **differenziatore chiave**.

TIPO REGOLA	FUNZIONE	ESEMPIO
Layout	Assegna catalogo + stile a sedi target	"Menu Pranzo con Stile Estate a tutte le sedi in città"
Visibilità	Nasconde o disabilita prodotti specifici	"Nascondi 'Ostriche Fresche' il lunedì"
Prezzo	Sovrascrive i prezzi per promozioni	"Tutti i cocktail €5 dalle 17–19, mostra prezzo originale barrato"

> **Modalità temporali:** `always` (sempre attivo), `daily` (finestra oraria ricorrente), `weekly` (giorni specifici + orario), `date_range` (basato su evento/periodo).

Stili / Temi (Styles)

Un sistema di theming basato su token che controlla il rendering dei cataloghi. Decoupled dai cataloghi — lo stesso catalogo può apparire diversamente a seconda dello stile attivo.

- **Versionamento immutabile:** Ogni modifica crea una nuova `style_version`. Rollback immediato.
- **Token:** navigazione (pill/tabs/minimal), card image (show/hide + posizione), layout (grid/flex)
- **Stili di sistema:** `is_system = true`, forniti dalla piattaforma, non eliminabili
- **Tracking utilizzo:** `usage_count` calcolato da referenze in `schedule_layout`

SEZIONE 04

Come funziona il sistema

Come si costruisce un catalogo

1. **Crea prodotti** nel database globale con varianti, gruppi opzione, attributi, allergeni, ingredienti e immagini.
2. **Crea un catalogo** e definisci la gerarchia di categorie (fino a 3 livelli).
3. **Aggiungi prodotti alle categorie** tramite l'editor split-pane del CatalogEngine. Drag-and-drop per l'ordinamento.
4. Il catalogo è ora un pacchetto di contenuto strutturato e riutilizzabile — ma non è ancora visibile.

Come il contenuto raggiunge una sede

1. **Crea un'attività** (sede) con nome, indirizzo e slug.
2. **Crea una regola di scheduling** (tipo: layout) che assegna catalogo + stile alla sede.
3. **Imposta la modalità temporale** (es. daily 11:00–15:00) e attiva la regola.
4. Il catalogo è ora **live** in quella sede durante quelle ore.

Come lo scheduling sovrascrive il comportamento

1. **Regole di visibilità** nascondono prodotti specifici in sedi/orari specifici.
2. **Regole di prezzo** sovrascrivono i prezzi per promozioni (happy hour, sconti, ecc.).
3. **Contenuti in evidenza** iniettano blocchi promozionali (hero banner, bundle) in slot specifici.
4. Le regole sono valutate per **priorità** — quelle con priorità più alta vincono i conflitti.

Come viene generata la pagina pubblica

1. Un cliente visita `/collections/{slug}` .

2. `resolveActivityCatalogs(activityId)` esegue: recupera regole attive → valuta condizioni temporali → seleziona la regola layout vincente → determina catalogo + stile.

3. Applica override di visibilità (filtra prodotti nascosti) e override di prezzo (calcola `effective_price`).

4. Risolve slot dei contenuti in evidenza.

5. Restituisce `ResolvedCollections` : `{catalog, style, featured}` .

6. `CollectionView` renderizza: header brand, hero, navigazione sezioni (sincronizzata con scroll), card prodotti stilizzate, modale dettaglio prodotto.

SEZIONE 05

Funzionalità chiave

Distribuzione cataloghi time-aware

I cataloghi si alternano automaticamente basandosi su giorno, ora o intervallo di date. Una sede può servire menu diversi durante la giornata senza azione dell'operatore.

Override granulari per prodotto

Senza toccare il database prodotti, le regole possono nascondere, disabilitare o cambiare il prezzo di specifici articoli — per sede, per finestra temporale.

Versionamento stili immutabile

Ogni modifica crea una nuova versione. Rollback immediato. Regole multiple possono referenziare versioni diverse dello stesso stile.

Catalog Engine gerarchico

Alberi di categorie a 3 livelli con drag-and-drop, guardie anti-duplicazione ed editor split-pane. I prodotti sono aggiunti una volta e referenziati ovunque.

Slot contenuti in evidenza

Blocchi promozionali (hero, before/after-catalog) con pricing flessibile (editoriale, somma per item, bundle fisso). Iniettati via regole di scheduling.

Targeting multi-sede

Le regole possono targetizzare sedi individuali, gruppi di sedi o tutte le sedi. I gruppi attività permettono "applica a tutte le sedi in città" senza elencarle una per una.

Sistema varianti e opzioni

Prodotti base con varianti (es. taglie), gruppi PRIMARY_PRICE (prezzo assoluto per taglia) e gruppi ADDON (prezzo delta per modificatore). `fromPrice` mostra "da X€".

Multi-tenancy sicura

PostgreSQL RLS con `get_my_tenant_ids()`. SECURITY DEFINER RPC per operazioni sensibili. Soft-delete con 30 giorni di recupero.

Autenticazione OTP

Due livelli: sessione Supabase + verifica OTP con hashing SHA-256, lockout brute-force (15 min) e record di verifica session-scoped.

Importazione menu con AI

Edge function `menu-ai-import` con integrazione OpenAI per parsing di menu esistenti in dati prodotto strutturati — accelera l'onboarding.

SEZIONE 06

Architettura e insight tecnici

Isolamento multi-tenant come primitiva del database

L'isolamento tenant non è una responsabilità applicativa — è una **primitiva PostgreSQL**. Ogni tabella tenant-scoped ha policy RLS che usano `get_my_tenant_ids()`, una funzione STABLE SECURITY INVOKER che si cache all'interno di una statement. Le funzioni di servizio fanno double-check con `.eq("tenant_id", tenantId)` come difesa in profondità.

> **Implicazione:** Anche un bug frontend non può far trapelare dati cross-tenant.

Scheduling come livello di orchestrazione

La maggior parte delle piattaforme usa flag booleani (`is_active`, `is_visible`) sulle singole entità. CataloGlobe usa **regole di scheduling come piano di controllo separato**:

- Il contenuto (prodotti, cataloghi) è **stateless** — non sa quando o dove viene mostrato.
- Le regole di scheduling **possiedono tutta la logica di distribuzione**.
- Questa separazione permette di autore il contenuto indipendentemente dalla distribuzione.

Disaccoppiamento catalogo-stile

I cataloghi **non hanno** una colonna `style_id`. Il binding tra catalogo e stile avviene **nella regola di scheduling** (`schedule_layout.catalog_id + style_id`).

> **Questo significa:** Lo stesso catalogo può renderizzare con stili diversi in sedi diverse e in orari diversi. Un cambio tema stagionale è un cambio alla regola, non una migrazione del catalogo.

Versionamento immutabile degli stili

Le config degli stili non vengono mai mutate in-place. Ogni modifica crea una nuova `style_version` con numero versione incrementale. L'entità stile punta a `current_version_id`. Questo fornisce: audit trail completo, rollback a rischio zero, e capacità futura di pin su versione specifica.

Riuso contenuti via database prodotti condiviso

I prodotti esistono una volta e sono referenziati ovunque. Una "Pizza Margherita" con varianti, opzioni, allergeni e immagini può apparire in "Menu Pranzo", "Menu Cena", "Catalogo Catering" e "Speciale Weekend" — tutto per riferimento. Un aggiornamento al prezzo base si propaga istantaneamente (a meno che non sia sovrascritto da una regola).

Accesso pubblico senza buchi nel RLS

Le pagine catalogo pubbliche non autenticano. Invece di aprire falle nel RLS, la piattaforma usa **RPC SECURITY DEFINER** (`get_tenant_public_info` , `resolveActivityCatalogs`) che espongono solo i dati esatti necessari.

Edge Functions per operazioni sensibili

Le operazioni distruttive (delete/restore/purge tenant, eliminazione account) sono gestite da Supabase Edge Functions che validano ownership via JWT, eseguono con privilegi `service_role` e gestiscono cascade delete atomiche (storage + tutte le tabelle figlie).

SEZIONE 07

Cosa rende speciale CataloGlobe

Non è un menu builder — è un content distribution engine

L'insight fondamentale è che **un catalogo digitale non è un documento statico**. È una **funzione del contesto**: quali prodotti esistono, come sono organizzati, che stile indossano, quale sede servono e che ora è.

La maggior parte degli strumenti tratta un menu come un file da progettare ed esportare. CataloGlobe lo tratta come un **output computato** di un motore a regole che opera su un database di contenuti condiviso.

L'astrazione dello scheduling cambia tutto

Rendendo lo scheduling un'entità first-class tra contenuto e distribuzione, CataloGlobe abilita comportamenti impossibili nei menu builder tradizionali:

- **Dayparting automatico:** Menu pranzo → bevande pomeridiane → menu cena → snack notturni, nella stessa sede fisica.
- **Promozioni location-aware:** Un'happy hour si applica solo alle sedi del centro, nei giorni feriali.
- **Flessibilità stagionale:** Passare da "Menu Estate" a "Menu Autunno" cambiando un date range, non ricostruendo cataloghi.
- **Potenziale A/B testing:** Due regole layout con stili diversi su gruppi di sedi diversi = A/B test visuale.

Separazione delle responsabilità come filosofia di prodotto

La maggior parte dei SaaS confonde contenuto, struttura, presentazione e distribuzione. CataloGlobe li **separa in livelli componibili**. È lo stesso insight architetturale che ha reso potente il CSS (separazione del contenuto dalla presentazione) — applicato alla gestione di cataloghi digitali.

Enterprise-ready dal giorno uno

L'architettura multi-tenant con PostgreSQL RLS, accesso role-based (owner/admin/member), onboarding basato su inviti, verifica OTP e soft-delete con periodo di grazia non è tipica di un SaaS in fase iniziale. Questa fondazione supporta lo scaling da un singolo ristorante a una rete di franchising senza cambi architetturali.

SEZIONE 08

Rischi e aree incomplete

Gap di sicurezza: v2_schedule_targets

La tabella `v2_schedule_targets` non ha colonna `tenant_id` né policy RLS. Il join a `v2_schedules` (che ha RLS) fornisce protezione indiretta, ma una query diretta potrebbe esporre dati di targeting cross-tenant.

Nessuna strategia di cache per le pagine pubbliche

La funzione `resolveActivityCatalogs` (54KB — il modulo di servizio più grande) esegue query live ad ogni caricamento. Non c'è layer di caching, CDN integration o strategia di pre-rendering. Per sedi ad alto traffico, questo potrebbe diventare un collo di bottiglia.

Sistema token stili minimale

I token attuali controllano forma navigazione, posizione immagine card e layout. Non ci sono token per colori, tipografia o spaziatura. L'architettura supporta personalizzazione ricca ma la copertura effettiva è limitata.

Analytics e Recensioni non implementati

La sidebar ha voci di navigazione "Analytics" e "Recensioni", le route esistono, ma non c'è servizio analytics né raccolta dati visibile nel codebase. Sono placeholder.

Interazione Featured Content + Scheduling

I contenuti in evidenza hanno `status` (draft/published) ma la loro iniezione nelle pagine pubbliche dipende dalle regole di scheduling. L'interazione tra lo stato del contenuto e quello della regola può creare edge case confusi (es. contenuto pubblicato referenziato da regola disabilitata).

Complessità di resolveActivityCatalogs

A 54KB, è di gran lunga il modulo più grande. Gestisce risoluzione catalogo, valutazione schedule, filtraggio visibilità, override prezzi e assembly contenuti in evidenza in un singolo file. Candidato per decomposizione.

Confine Edge Functions inconsistente

Alcune operazioni usano edge functions (tenant delete/restore/purge, business delete) mentre operazioni simili usano chiamate di servizio dirette. Il confine di "cosa necessita un'edge function" non è sempre chiaro.